

SIMD&SIMT新同构编程系列： 算子性能优化经验分享

聚焦昇腾 NPU 算子性能优化 · 三大实践案例

■ 本次分享 · 三大实践案例

从访存密集型 Vector 算子，到 Scalar 瓶颈定位，再到 MXFP8/MXFP4 混合精度矩阵乘，覆盖算子优化的关键路径

01 Part 1

157×

RmsNormQuant 性能深度优化

Vector · 访存密集型算子逐级优化

总加速比 7693us → 49.0us

02 Part 2

33%~50%

Scalar 算子性能优化

Scalar 如何影响昇腾 NPU 算子性能

Scalar 耗时平均降低 · 端到端提升 10%~15%

03 Part 3

快 38.2%

**MXFP8/MXFP4 混合精度矩阵乘
优化**

Vector + Cube 混合核 · 位操作优化

M=1 kernel 17.4us vs 28.2us

目录

CONTENTS

Part 1 RmsNormQuant 性能深度优化

Part 2 Scalar 算子性能优化

Part 3 MXFP8/MXFP4 混合精度矩阵乘优化

RmsNormQuant

性能深度优化

Ascend 950 上的高性能 Vector 算子分阶段优化实践

157×

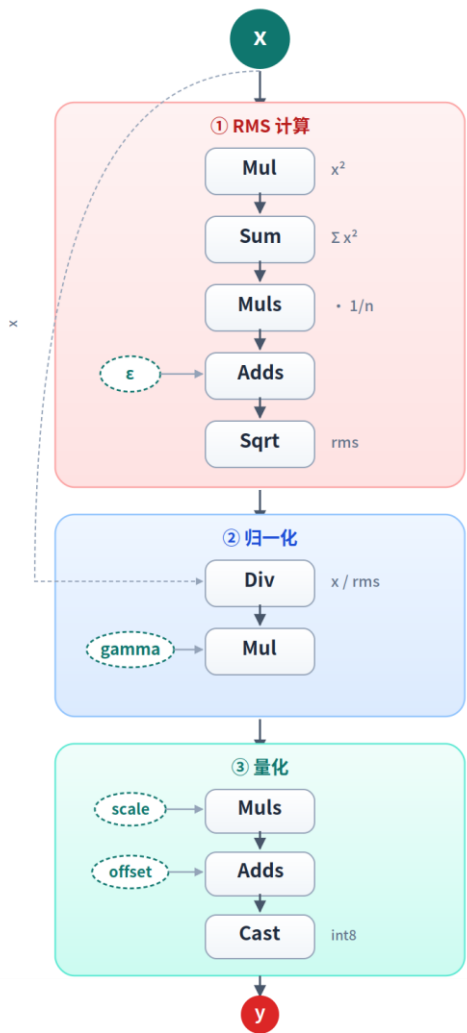
7693 us → 49.0 us

总加速比 · 单核 7693us 优化至 64 核 49us

Ascend 950 · x[4096, 8192] fp16 → y int8

RmsNormQuant 算子介绍与优化目标

计算图



157x

7693 us $\rightarrow$ 49.0 us

Ascend 950 · x[4096, 8192] fp16 $\rightarrow$ y int8

算子功能

RmsNormQuant 是 LLM 推理中 **RmsNorm 归一化** 与 **Int8 量化** 的融合算子。

通过消除中间结果的 GM 写回 + 读入，相比分离执行在访存效率上具有显著优势，是优化 Vector 算子的典型场景。

三个核心特点:

- ① 行间独立 — 每行 RMS 仅依赖本行数据，天然适合多核并行
- ② 行内长归约 — hidden_size = 8192，列方向不能随意切碎
- ③ gamma 跨行共享 — 4096 行复用同一份 16 KB 权重

计算公式

$$RMS_i = \sqrt{(1/r) \cdot \sum_j x_{ij}^2 + \epsilon)}$$

$$norm_{ij} = x_{ij} / RMS_i \cdot \gamma_j$$

$$y_{ij} = clamp(round(norm_{ij} \cdot s + b), -128, 127)$$

Step 1: Gamma 预加载 — 从公式入手识别冗余搬运

17693 → 6790 us · 1.13×

公式推导路径 · 本步无需 profile 观测，从数学结构静态分析即可识别冗余

$RMS_i, norm_{ij} \cdot \gamma_j \rightarrow \gamma$ 下标只有 j $\rightarrow$ 跨行共享 / 只读 / 固定 16 KB $\rightarrow$ 外提循环、常驻 UB

✓ 验证 · 指标变化

- MTE2 30.2% $\rightarrow$ 28.3%
- Duration 7693 us $\rightarrow$ 6790 us
- 单步加速 1.13×

① 从公式入手 · 识别 γ 的数据特征

$$RMS_i = \sqrt{(1/r) \cdot \sum_j x_{ij}^2 + \varepsilon}$$
$$norm_{ij} = (x_{ij} / RMS_i) \cdot \gamma_j \leftarrow \gamma \text{ 只有下标 } j$$

- ▶ 跨行共享: γ_j 仅依赖列索引 j, 4096 行复用同一份
- ▶ 只读: 整个 kernel 周期内 γ 不被修改
- ▶ 大小固定: $hidden_size \times 2B = 8192 \times 2 = 16 \text{ KB}$

② 推论 · 把 γ 搬运提到循环外

✗ 原始代码

```
// 0_naive.cpp
__aicore__ inline void Process() {
    for (int64_t loop = 0;
        loop < tilingData_ ->a; loop++) {
        CopyInGamma(); // 冗余
        CopyInX(loop);
        Compute();
        CopyOut(loop);
    }
}
```

✓ 优化后

```
// 1_preload_gamma.cpp
__aicore__ inline void Process() {
    CopyInGamma(); // 仅 1 次
    for (int64_t loop = 0;
        loop < tilingData_ ->a; loop++) {
        CopyInX(loop);
        Compute();
        CopyOut(loop);
    }
}
```

③ 跷跷板权衡 · 预加载并非零代价

✓ 收益

省去 ~64 MB 冗余搬运

4095 次循环 $\times$ 16 KB $\approx$ 64 MB MTE2 搬运被消除

✗ 代价

占用 UB $\approx$ 49 KB

γ fp16 搬入缓冲 + fp32 工作缓冲 $\approx$ 19% UB

适用条件

共享数据 $\ll$ UB 容量

本例 γ 16KB $\ll$ 256KB UB, 明确收益侧

Step 2: 多核并行 — 利用行间独立性激活 64 核

2 6790 → 113.6 us · 59.8×

观测 · Profile 指标

- Block Num **1**
- 带宽 **0.01 TB/s**
- 空闲核数 **63 / 64**

推导 · 优化手段

算子特性 · 并行推导

算子特性「行间独立」→ M 轴可任意切分，沿 M 均分到 64 核并行执行。

验证 · 指标变化

- Block 1 → **64**
- 带宽 0.01 TB/s → **0.59 TB/s**
- Duration 6790 us → **113.6 us**

① 从算子特性推导 · 行间独立

$$RMS_i = \sqrt{((1/r) \cdot \sum_j x_{ij}^2 + \varepsilon)} \quad \text{仅依赖第 } i \text{ 行}$$

- ① 无跨行依赖：每行 RMS / norm / quant 完全独立计算
- ② 无核间同步：切到哪一行都不需要其它核的中间结果
- ③ 切分自由：M 轴可任意切分；本例 4096 / 64 = 64 行/核

② 代码改动 · 核心 3 行

```
// Kernel Init()
blockIdx_ = AscendC::GetBlockIdx();
xGm_.SetGlobalBuffer(x + blockIdx_ * blockFactor * r);
yGm_.SetGlobalBuffer(y + blockIdx_ * blockFactor * r);

// Host launch
kernel<<<64, 0, stream>>>(...);
```

► 并行效率：6790 / 113.6 = **59.8×** · 全核利用率 **93.4%** (接近线性加速)

Step 2 流水打点图：64 核并行执行，各核独立处理不同的 token 行



💡 **关键洞察：**拿到性能差的算子，第一步永远是看核数。多核并行不需要硬件深度知识，却往往贡献了全流程最大的加速。

Step 3: 寄存器数据流 (VF RegAPI) 压缩计算链路

3113.6 → 84.1 us · 1.35×

观测 · Profile 指标

- 主导流水 **VEC 63%**
- IPC **1.1**
- Duration **113.6 us**

推导 · 优化手段

VF RegAPI

中间值常驻向量寄存器，打破 UB↔UB 抽象；以 reduce 为天然边界拆 vf1/vf2；rms 随路 broadcast 进 vf2。

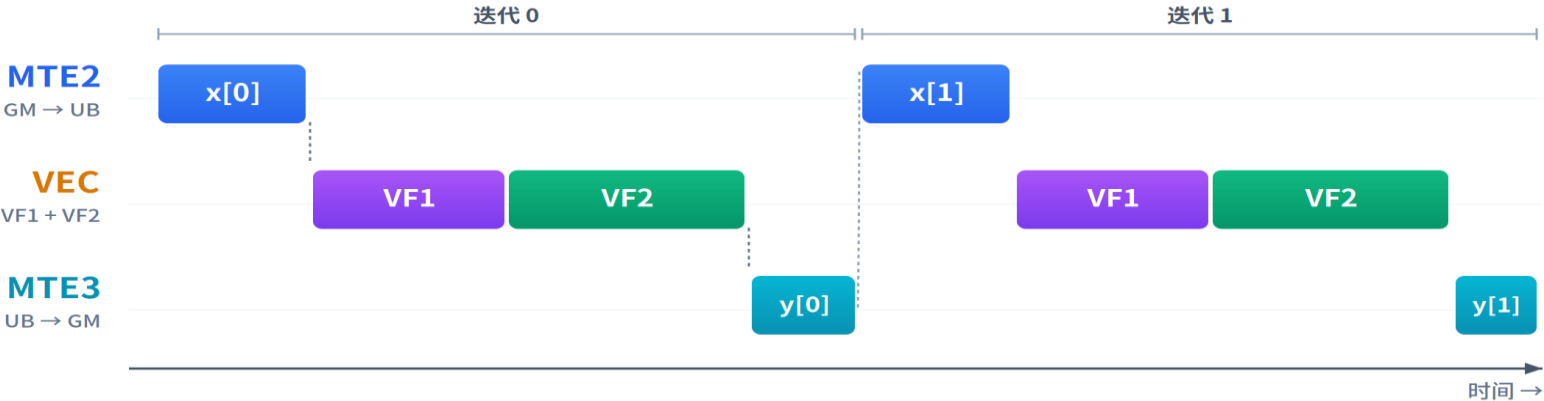
验证 · 指标变化

- VEC 63% → **49.4%**
- MTE2 31.3% → **45.3%**
- IPC 1.1 → **1.34**

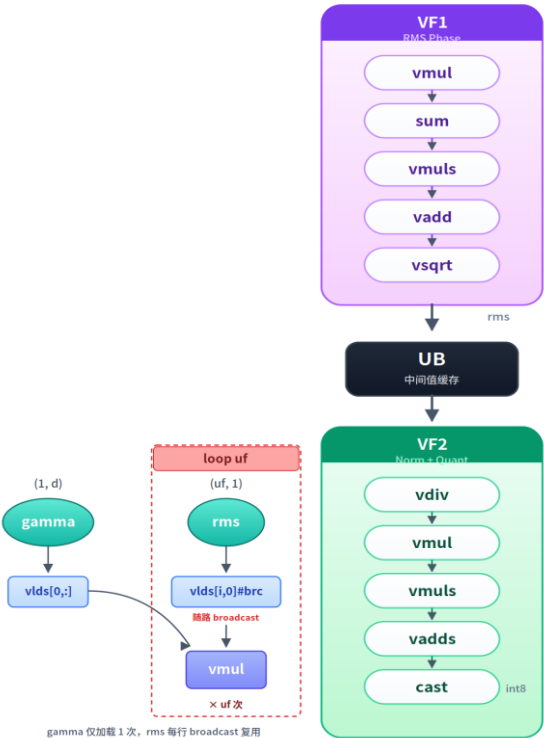
VF 性能优化

- ▶ **VF 融合**：以 reduce 节点为界拆分为两个 VF 执行 — vf1：均方根计算 (rms)，vf2：归一化 + 量化 (norm + quant)。当前支持手写融合、毕昇编译器自动识别、FlashVF 工具寻优。
- ▶ **随路 Broadcast**：将计算后的 rms 值随路 broadcast 到 vf2 中。
- ▶ **本质**：打破 UB↔UB 抽象，中间值始终驻留向量寄存器，仅循环结束后写回 UB 一次。

Step 3 流水时序：VF1 + VF2 串行



⚠ 迭代之间严格串行 — MTE2、VEC、MTE3 不重叠，VEC 占主导



Step 3 (续): VF RegAPI 真实代码 — 两个 `__simd_vf__` 函数

3 源自 3_vf.cpp · *ComputeRmsVf*
/ *ComputeNormQuantVf*

▶ VF 核心特征: ① `__simd_vf__` 标记可融合段 ② `Reg::RegTensor / MaskReg` 驻留向量寄存器 ③ **DIST_BRC_B32** 随路 broadcast 替代 UB 中转

vf1 • ComputeRmsVf (RMS 阶段)

```
__simd_vf__ inline void ComputeRmsVf(
    __ubuf__ DATA_TYPE *xInAddr,
    __ubuf__ float *rmsAddr) {
    Reg::RegTensor<DATA_TYPE> vregXIn;
    Reg::RegTensor<float> vregX,
        vregXQuared, vregReduceSum, vregRms;
    Reg::MaskReg preg = Reg::CreateMask<float>();
    Reg::Duplicate(vregReduceSum, 0);

    // 行内累加:  $\sum(x^2)$ 
    for (uint16_t i = 0; i < vfLoopRNum_; i++) {
        preg = Reg::UpdateMask<float>(r);
        Reg::DataCopy<DATA_TYPE, DIST_UNPACK_B16>(
            vregXIn, xInAddr + i * VL_B32_SIZE);
        Reg::Cast<float, DATA_TYPE, ...>(vregX, vregXIn, preg);
        Reg::Mul(vregXQuared, vregX, vregX, preg);
        Reg::Add(vregReduceSum, vregReduceSum,
            vregXQuared, pregAll);
    }

    //  $rms = \sqrt{\text{mean}(x^2) + \epsilon}$ 
    Reg::ReduceSum(vregReduceSum, vregReduceSum, preg);
    Reg::Muls(vregRms, vregReduceSum, rlnv_, preg);
    Reg::Adds(vregRms, vregRms, tilingData_ -> epsilon, preg);
    Reg::Sqrt(vregRms, vregRms, preg);
    Reg::DataCopy<float, DIST_FIRST_ELEMENT_B32>(
        rmsAddr, vregRms, preg);
}
```

vf2 • ComputeNormQuantVf (Norm + Quant 阶段)

```
__simd_vf__ inline void ComputeNormQuantVf(
    __ubuf__ DATA_TYPE *xInAddr,
    __ubuf__ float *gammaAddr,
    __ubuf__ float *rmsAddr,
    __ubuf__ OUTPUT_DTYPE *yAddr) {
    Reg::RegTensor<float> vregX, vregGamma,
        vregRms, vregNorm;
    Reg::RegTensor<half> VregYFp16;
    Reg::RegTensor<OUTPUT_DTYPE> VregY;

    // ★ 随路 broadcast 载入 rms (vf1 的结果)
    Reg::DataCopy<float, DIST_BRC_B32>(
        vregRms, rmsAddr);

    for (uint16_t i = 0; i < vfLoopRNum_; i++) {
        preg = Reg::UpdateMask<float>(r);
        Reg::DataCopy<DATA_TYPE, DIST_UNPACK_B16>(
            vregXIn, xInAddr + i * VL_B32_SIZE);
        Reg::Cast(vregX, vregXIn, preg);
        Reg::DataCopy(vregGamma, gammaAddr + i * VL);
        Reg::Div(vregNorm, vregX, vregRms, preg);
        Reg::Mul(vregNorm, vregNorm, vregGamma, preg);
        Reg::Muls(vregNorm, vregNorm, scale_, preg);
        Reg::Adds(vregNorm, vregNorm, offset_, preg);

        // fp32 → fp16 → int8 量化
        Reg::Cast<half, float, ...>(VregYFp16, vregNorm, preg);
        Reg::Cast<OUTPUT_DTYPE, half, ...>(VregY, VregYFp16, preg);
        Reg::DataCopy<OUTPUT_DTYPE, DIST_PACK4_B32>(
            yAddr + i * VL, VregY, preg);
    }
}
```

Step 4: Double Buffer 实现流水重叠

484.1 → 54.3 us · 1.55×

观测 · Profile 指标

- VEC 49.4%
- MTE2 45.3% (≈ 1:1)
- V+M 94.7% — 串行模式

推导 · 优化手段

Double Buffer

对输入 x 申请双 buffer 轮流使用 — 当前轮 VEC 算 buffer A，下一轮 MTE2 同时搬 buffer B。

验证 · 指标变化

- V+M 94.7% → 161.7%
- 主导流水 49.4% → 83% (破 80%)
- Duration 84.1 us → 54.3 us

流水优化策略

► Double Buffer: 对输入 x 使用双缓冲 buffer，当前轮 VEC 计算时下一轮 MTE2 同时搬运，实现流水重叠。

► 其他 buffer: gamma 继承 Step 1 优化，scale/offset 启动期一次搬入，均不参与双缓冲，避免打断流水。

► 代码改动 · 仅 1 行 (BUF_NUM: 1 → 2)

```
// 3_vf.cpp:61
static constexpr size_t BUF_NUM = 1;
// 4_double_buffer.cpp:61
static constexpr size_t BUF_NUM = 2;
```

✗ Before · 串行流水 MTE2 → VEC → MTE3 每拍仅一条流水工作

✓ After · 重叠流水 (Double Buffer) 三条流水并行推进



Task Duration	MTE2 带宽	IPC	VEC + MTE2	主导流水
54.3 us	1.24 TB/s	1.34	161.7% 重叠模式	83% (VEC) ≥ 80% 达标

✓ VEC + MTE2 = 161.7% >> 100% — 流水正式进入重叠模式，MTE2 搬运被 VEC 计算掩盖



Step 5: UB 多行批处理 + Tiling 空间建模

554.3 → 49.0 us · 1.11×

观测 · Profile 指标

- 带宽 1.24 / 1.6 TB/s (70%)
- IPC 1.34 / 2.0 (上限)
- ubFactor 1 (循环 64 次/核)

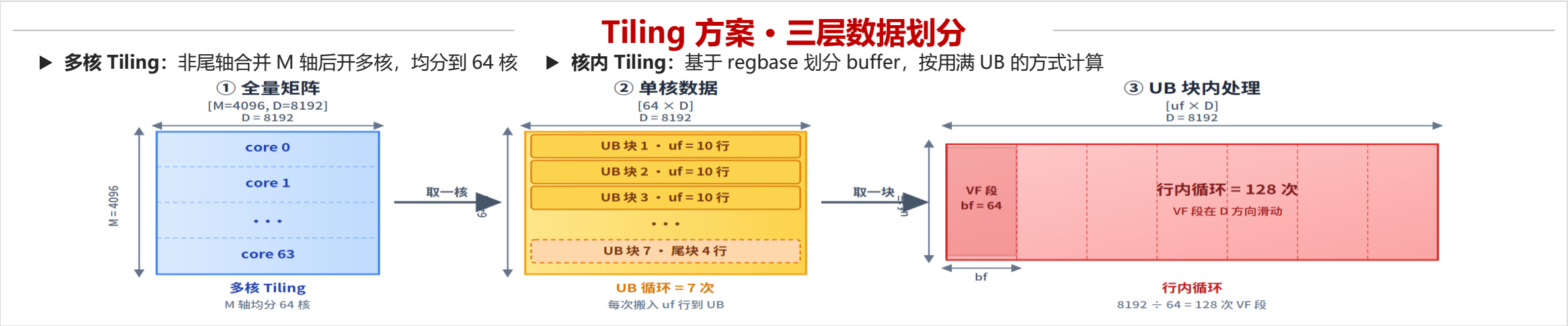
推导 · 优化手段

UB 空间建模

UB_total = fixedSize + uf × linearCoef 反解
maxUbFactor ≈ 10。

验证 · 指标变化

- 循环次数 64 → 7 次
- 带宽 1.24 TB/s → 1.37 TB/s
- IPC 1.34 → 1.42



UB 空间建模

$$UB_total = fixedSize + uf \times linearCoef$$

→ 求解得 maxUbFactor ≈ 10

缓冲区	大小	类型
gammaBuf	rAlign × 4B	固定
xInQueue	rAlign × 2B × 2 × uf	线性
yOutQueue	rAlign × 1B × 2 × uf	线性
rmsBuf	4B × uf	线性

三层循环结构 (ubFactor ≈ 10)

核间并行
blockIdx · 64 核
= 4096 行
64 × 64 行 全行覆盖

UB 循环
[64 / 10]
= 7 次
最后一次 4 行

行内循环
[8192 / 64]
= 128 次
VF 向量段

指标演进总览：性能与关键指标同步推演

全流程关键指标演进 (从 CSV 实测)

📁 数据来源: cannsim profiler CSV (0_naive.csv ~ 5_ub_utilization.csv) · 各步取有效行中位数

Step	Dur	×倍	Block	BW	IPC	VEC%	V+M%
0 Naive	7693	1×	1	0.01	1.1	66.9	97.1
1 Preload γ	6790	1.1×	1	0.01	1.1	68.5	96.8
2 Multi-core	114	67.7×	64	0.59	1.1	63.0	94.3
3 VF RegAPI	84.1	91.5×	64	0.8	1.34	49.4	94.7
4 Double Buf	54.3	141.7×	64	1.24	1.34	83.0	161.7
5 UB Batch	49.0	157×	64	1.37	1.42	82.9	156.7

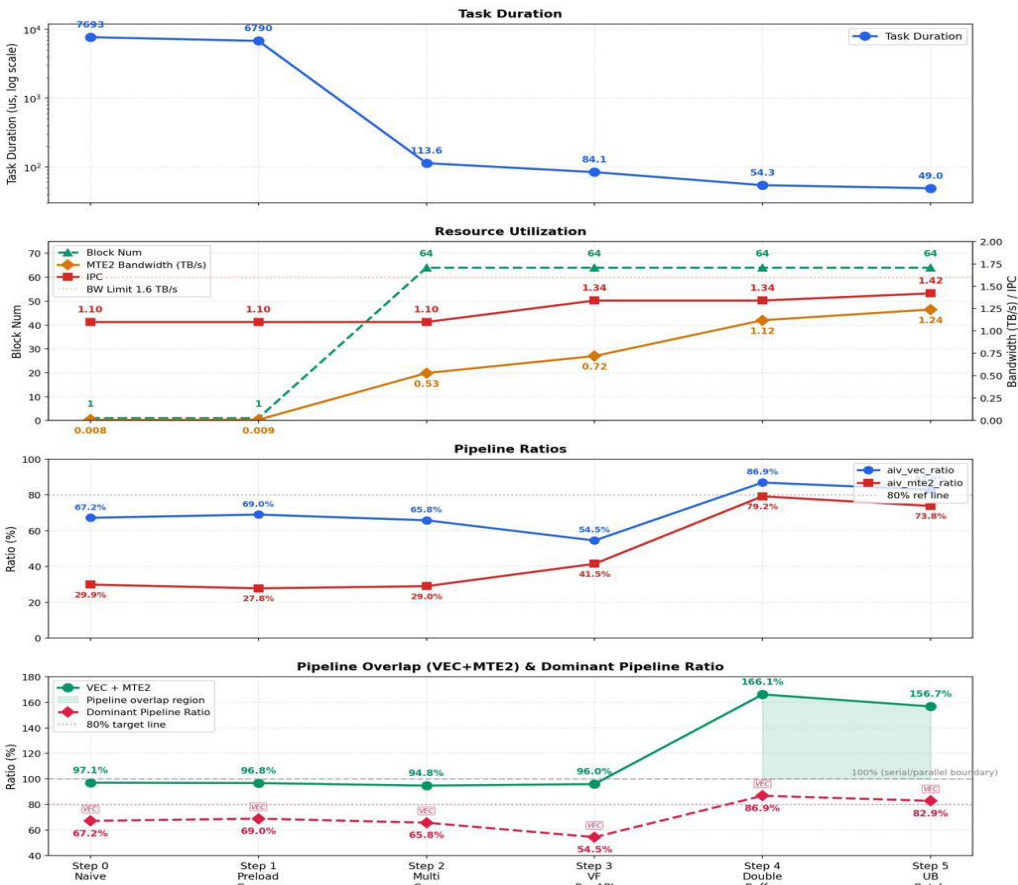
每一步背后的指标观测:

- ▶ Step 2 Block 1→64 → 揭示资源闲置，多核并行单步 59.8×
- ▶ Step 3 VEC 63%→49.4% · IPC 1.1→1.34 → 单核效率瓶颈被压缩
- ▶ Step 4 V+M 94.7%→161.7% → 流水正式从串行 (<100%) 跳入重叠 (>>100%)
- ▶ Step 5 Duration 54.3→49.0 us · 累计 157×

性能演进图：耗时 + 资源 + 流水

Step 0~5 完整性能曲线

RmsNormQuant Performance Overview (Step 0 ~ Step 5)



🔗 数据驱动优化闭环：看 profile 找异常指标 → 推断瓶颈类型 → 选择优化手段 → 用同一组指标验证。157× 加速由 5 次这样的小循环堆叠而来。

问答

复现指南 · 工具链

三条命令跑通全流程

► 编译

```
cmake -S . -B build -DNPU_ARCH=dav-3510  
cmake --build build --target rms_norm_quant_story
```

► 仿真打点

```
cannsim record ./rms_norm_quant_<step> \  
-s Ascend950 --gen-report
```

► 流水可视化

Perfetto UI / chrome://tracing/ → 上传 trace_core0.json

代码 · 文档 · 系列文章



代码仓库 (cann-samples)

gitcode.com/cann/cann-samples
/Samples/2_Performance/rms_norm_quant_story



源码组织 (6 个独立可编译文件)

[src/0_naive.cpp](#) ~ [src/5_ub_utilization.cpp](#)



配套博客 · 更详细的实现讲解

README.md 含 step-by-step 性能优化指南



Q & A

• 欢迎在评论区或仓库 issue 提问 · 感谢大家的关注!

目录

CONTENTS

Part 1 RmsNormQuant 性能深度优化

Part 2 Scalar 算子性能优化

Part 3 MXFP8/MXFP4 混合精度矩阵乘优化

Scalar 如何影响昇腾 NPU 算子性能

—— 原理与优化实践

从 *ScalarBound* 现象 → *ScalarBound* 根因 → 两个典型算子优化实践 → 7 条编码原则

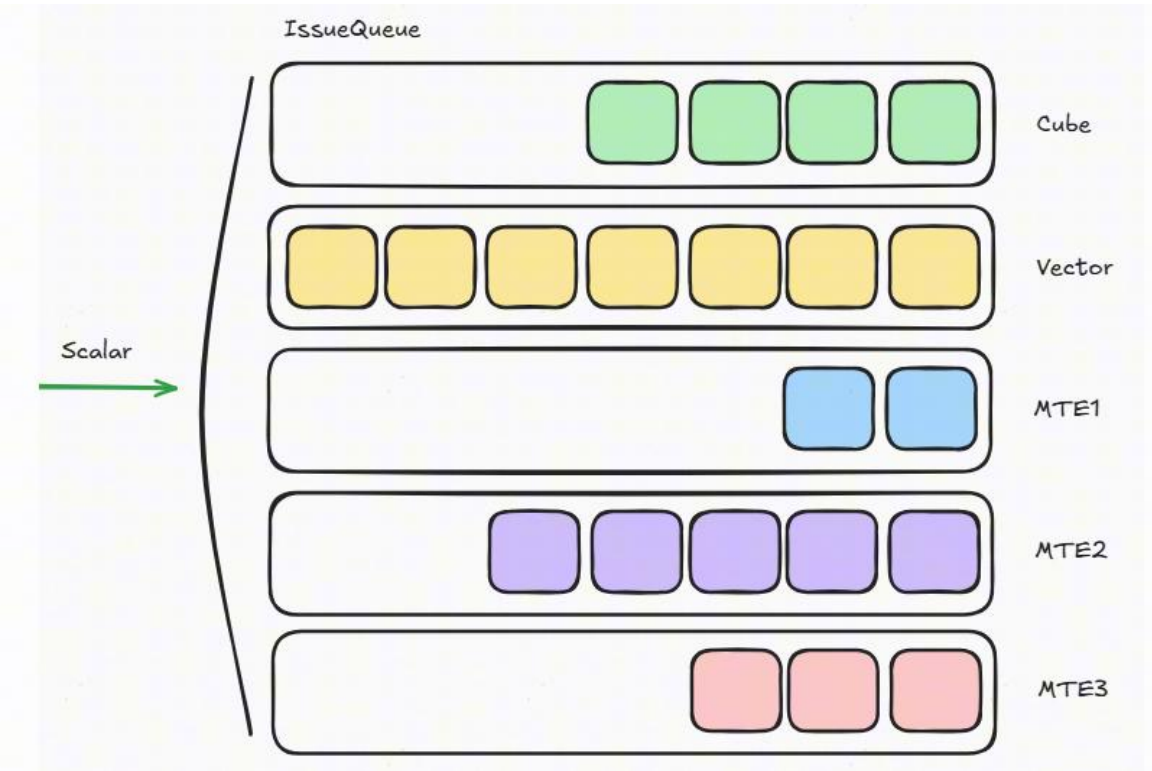
33%~50%

Scalar 耗时平均降低
端到端性能平均提升 10%~15%

AI Core 五大并行流水线

Cube Engine	矩阵乘加 (MMAD) — Matmul / Conv / FlashAttention 核心
Vector Engine	向量运算 — element-wise / Reduce
MTE1 / MTE2 / MTE3	数据搬运: $L1 \leftrightarrow L0$ / $GM \leftrightarrow L1 \cdot UB$ / $L1 \cdot UB \leftrightarrow GM$
FixPipe	$L0C \rightarrow$ 外部存储/UB, 格式转换与后处理
Scalar Unit	标量计算、地址计算、指令参数构造, 并为其他流水线分发指令

Scalar 指令分发示意

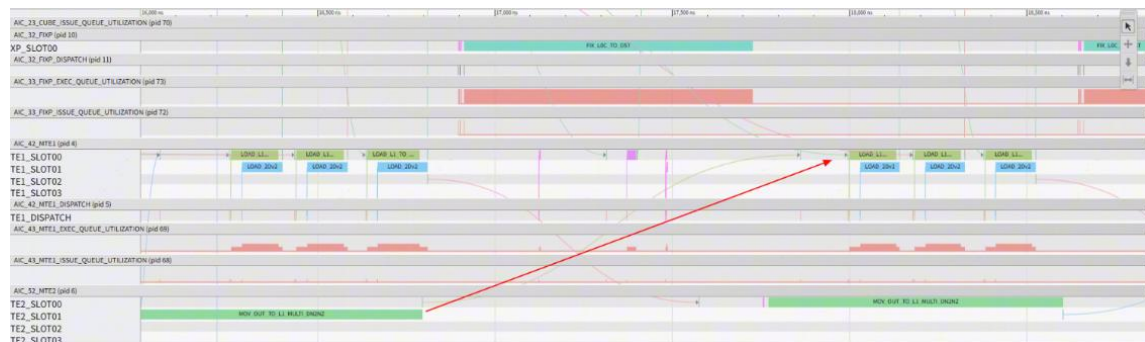


Scalar 是所有流水线的「指令分发中枢」 — 它一旦卡住, Cube / Vector / MTE 全部空转
反向影响: 任何流水线 Issue 队列满载, 也会反压回 Scalar, 阻塞后续指令分发。

什么是 ScalarBound? 为什么要关心?

定义 当 Scalar 单元成为瓶颈、无法及时为其他流水线分发指令，导致流水线产生 bubble（空闲周期）时，这种现象称为 ScalarBound。

危害 ① 流水中断，破坏指令级并行



Conv2D 示例：MTE2 与 MTE1 之间出现 >1000 cycle 的空泡，指令级并行严重受损。

危害 ② Scalar 自身高耗时直接拖慢算子

```
=====
kernel total ticks : 25397
system total ticks : 31561
=====
Trace_Name                | Total_Num
=====
fixp_cubecore0_to_l1_real_active_cycle | 0
fixp_cubecore0_busy_cycle   | 6742
cube.cube_cubecore0_busy_cycle | 11042
bwif_avg_bandwidth_mte      | 0
cube.cube_cubecore0_mac_busy_cycle | 9200
brif_mte_avg_bandwidth      | 50
mte1_cubecore0_su_busy_cycle | 7308
CCU.scalar_cubecore0_su_busy_cycle | 18155
fixp_cubecore0_to_out_real_active_cycle | 0
fixp_cubecore0_to_ub_real_active_cycle | 6737
mte2_cubecore0_su_busy_cycle | 12428
mte3_cubecore0_su_busy_cycle | 0
+++++core: 0, subcore: 0, start: 6164, tick: 31561, kernelId: 0, blkID: 0, blkDim: 64, rtsqid: 1, streamId:
```

BatchMatmul 示例：Scalar 耗时 18,155 cycle，远超其他所有流水线，成为单点最大开销。

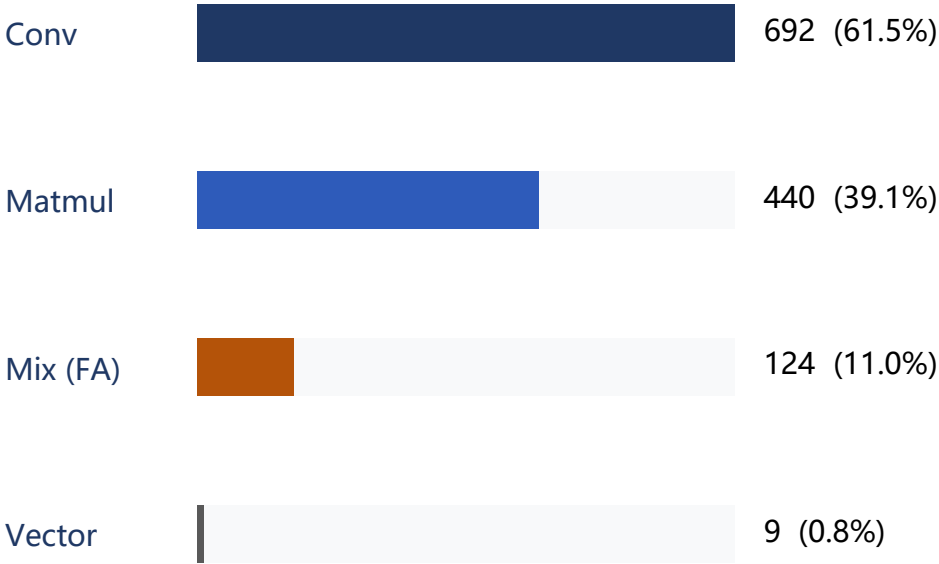
两类危害叠加：流水线被打散 + Scalar 本身贵 → 算子总耗时显著恶化

现状统计：谁是 ScalarBound 重灾区？

Top 10 ScalarBound 算子（按用例数）

Op Name	Count	类型
conv2dv2	160	Conv
mat_mul_v3	152	Matmul
conv3d_backprop_input_v2	142	Conv
quant_batch_matmul_v3	127	Matmul(量化)
conv3d_backprop_filter_v2	122	Conv
fused_infer_attention_score	96	Mix(FA)
batch_mat_mul_v3	70	Matmul
weight_quant_batch_matmul_v2	52	Matmul(量化)
quant_batch_matmul_v4	39	Matmul(量化)
conv2d_backprop_filter_v3	33	Conv

类型分布（共 1125 用例）



结论 FlashAttention + Matmul + Conv 占 **97.6%** (1098 / 1125) → Cube 类与 Mix 类算子是 Scalar 优化重点。
Vector 类不到 3%，本文暂不深入分析。

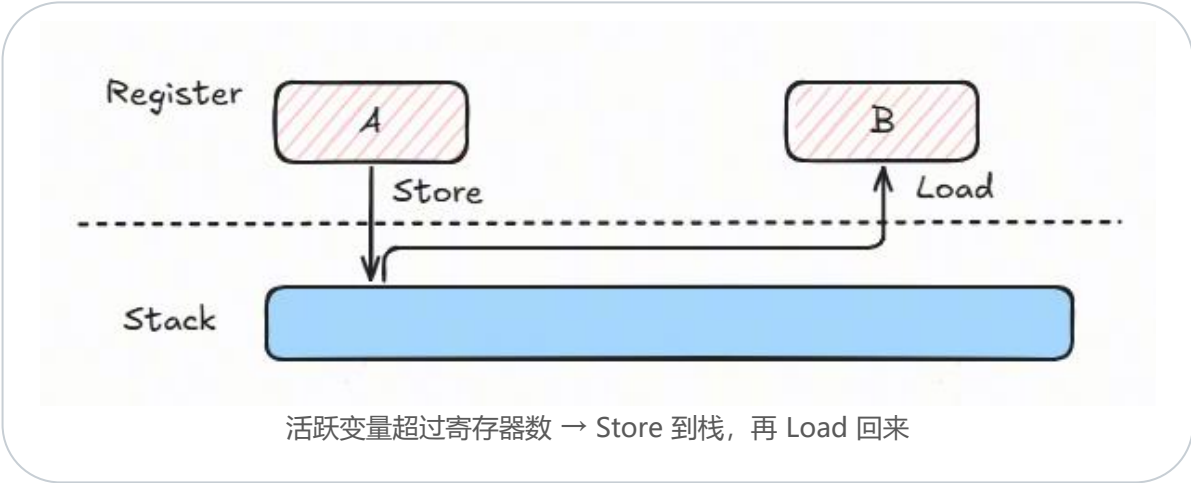
根因：Load/Store 过多与寄存器 Spill

ScalarBound 的三大成因

成因	描述	是否 Scalar 自身问题
Load 依赖阻塞	Scalar 等待 Load 指令的结果（如从栈加载）	是 ✓
计算指令过多	Scalar 指令数过多，分发跟不上消费速度	是 ✓
反压阻塞	其他 Issue 队列满，反压回 Scalar	一般不是

Cube/Mix 算子中 Load/Store 占比普遍 >30%，RAW 依赖中大部分 stall 来自等待 Load 完成

寄存器 Spill 原理



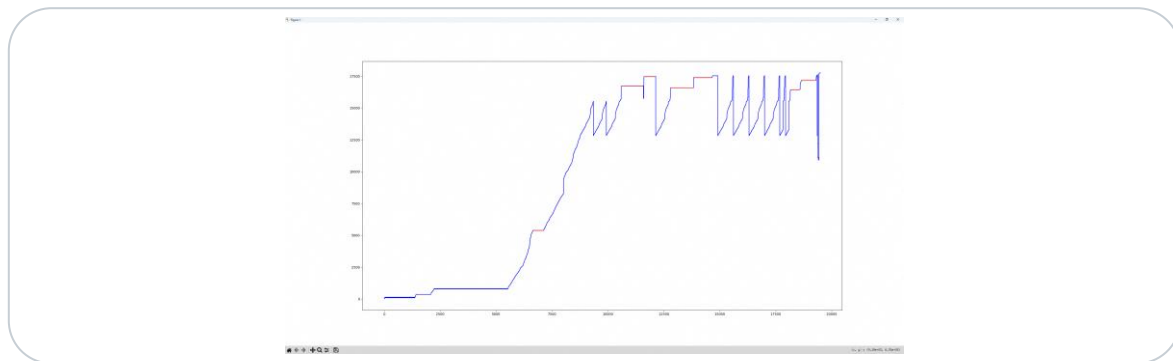
易触发 Spill 的编码：成员变量过多 / 结构体拷贝 / 变量生命周期长 / 多级指针解引用

实践 ①: FusedInferAttentionScore (上)

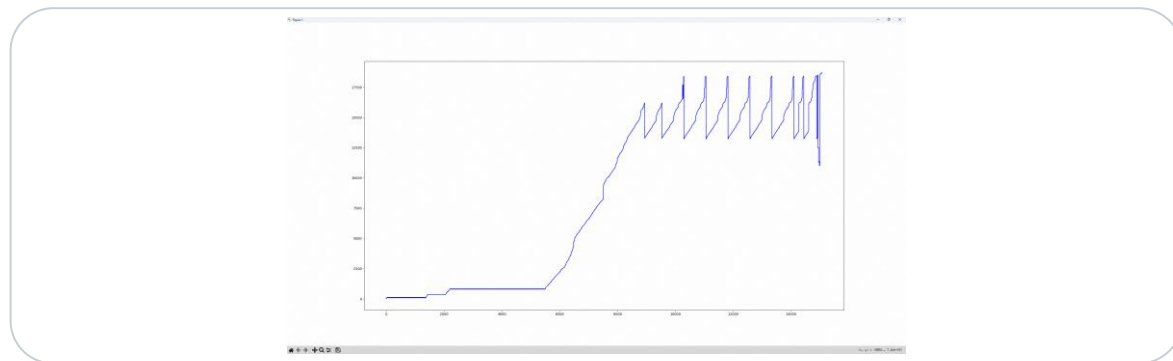
8 / 14

手段一 icache 预取 —— 消除 PC 等待, I-Cache miss 完全消失

优化前 (PC 出现长水平段 = I-Cache miss)



优化后 (预取指令插入后, 水平段消失)



手段二 静态创建 LocalTensor —— 跳过 TPipe 通用调度开销

优化前: 动态 InitBuffer / TBuf::Get

```
for (int32_t i = 0; i < L1KVNumSacler; ++i) {  
    pipe->InitBuffer(tmpBufL1KV[i], KV_L1_SIZE);  
    L1KVTensorPingPong[i] = tmpBufL1KV[i].template Get<T>();  
}  
  
pipe->InitBuffer(tmpBufL1Q, Q_L1_SIZE);  
L1QTensorPingPong = tmpBufL1Q.template Get<T>();  
// ... 含条件判断、队列操作等通用调度
```

优化后: 常量偏移直接构造 LocalTensor

```
L1KVTensorPingPong0 = LocalTensor<T>(  
    TPosition::B1, (Q_L1_SIZE + 2 * P_L1_SIZE) * 2, KV_L1_SIZE);  
  
L1KVTensorPingPong1 = LocalTensor<T>(  
    TPosition::B1, (Q_L1_SIZE + 2 * P_L1_SIZE + KV_L1_SIZE) * 2, KV_L1_SIZE);  
  
// 偏移 = 此前所有 buffer 大小之和 * 2 (ping-pong)  
// 编译期完全确定, 无运行时调度
```

实践 ①: FusedInferAttentionScore (下) + 效果

手段三 局部变量替代成员变量

constParam: 成员变量 → 局部变量

```
// 反面
class PromptFlashAttentionNormalBNS1Preload...{
protected:
    ConstParam constParam;    // 成员变量
};

// 正面
void Init() {
    ConstParam constParam;    // 函数内局部
    // ...
}
```

效果 Cube 核 Scalar 耗时 -15%，dcache miss 完全消除

8 组用例端到端对比 (节选关键列，单位: cycle)

#	优化前 TOTAL	优化前 SCALAR	优化后 TOTAL	优化后 SCALAR	SCALAR 降幅	TOTAL 提升
1	24408	14876	19890	9979	32.92%	18.51%
2	32400	19148	27432	14425	24.67%	15.33%
3	25182	13293	20250	9512	28.44%	19.59%
4	24210	14187	20268	11077	21.92%	16.28%
5	26208	12881	22212	8692	32.52%	15.25%
6	24354	14229	21240	9899	30.43%	12.79%
7	32166	16935	26820	8377	50.53%	16.62%
8	52704	24114	50832	13416	44.36%	3.55%

ScalarBound 完全消除

手段四 优化 tilingdata 拷贝

做法: 直接经入参指针读取，不再整块拷贝到栈

用例	优化前 (cycle)	优化后 (cycle)	降幅
bf16_8_128_4096_512_bsnd_rope	9060	4117	-54.6%
BF16_FP4_E2M1_BF16_64_8_4096_128_G QA...	8626	4426	-48.7%

效果 软件头开销直接 -50% 量级

平均效果

-33.22%

Scalar cycle 平均降低

+14.74%

端到端平均提升



实践 ②：QuantBatchMatmul (上)

手段一 局部变量提高数据访问局部性 —— 上帝类拆解为局部聚合

优化前：上帝类成员贯穿整个函数 → 易 Spill

```
class MatMulPerBlock {  
    // 大量成员变量集中保存所有参数  
    // 在函数入口很早就创建对象  
    // 生命周期贯穿整个函数  
    // → 编译器倾向于溢出到栈  
};
```

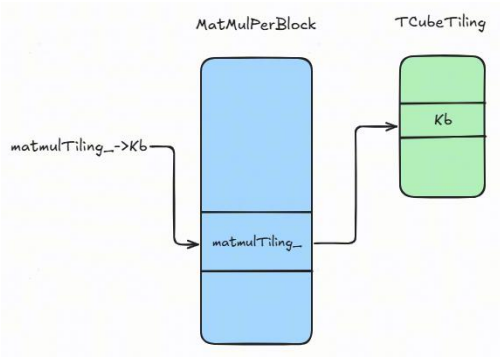
优化后：定义局部结构体保存计算所需参数

```
QbmmTiling qbmmParams{  
    tilingParamsIn->params.batchC,  
    tilingParamsIn->params.batchA1,  
    /* ... 拍平 ... */  
};  
Params params = { {1,1,1,1}, {x, weight, y, bias}, ... };  
QbmmKernel qbmm;  
qbmm(params);
```

手段二 消除 tiling 的多级指针解引用

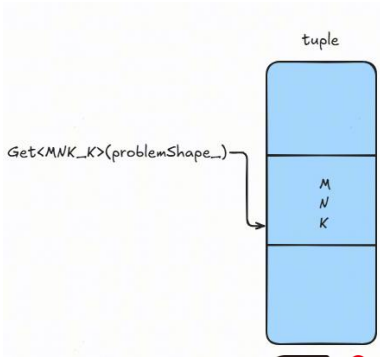
优化前：指针成员，每次访问 2 次 Load + 偏移加法

```
const TCubeTiling *matmulTiling_  
isTailBL1 = (kInner + minStepK_)  
    >= matmulTiling_->Kb;
```



优化后：值类型 tuple，1 次 Load 即可

```
using TupleShape = AscendC::Shape<  
    int64_t, int64_t, int64_t>;  
TupleShape problemShape_  
Get<MNK_K>(problemShape_);
```



每多一级指针 = 多一条 Load + 一条依赖链；值类型让编译器有信心做寄存器缓存



手段三 数组导致常量传播失效 —— 结构体内拿掉动态索引数组

```
1 template <typename T, bool enableSmallC0>
2 __aiore__ inline __inout_pipe__(MTE2) void DataCopy(const LocalTensor<T>& dst, const GlobalTensor<T>& src,
3             const Nd2NzParams& intriParams, bool enablePreReadIgnoreSync)
4 {
5     CheckNd2NzParams(intriParams, "DataCopy with Nd2NzParams");
6     using PrimType = Prim<T>;
7     const Hardware dstHWPos = GetPhysType((TPosition)dst.GetPosition());
8     ASCENDC_REPORT_OVERFLOW_MEM(CheckDataCopyTensorSizeOverflow(dst, src, intriParams));
9
10    // dav_c310 DataCopyGM2L1ND2NZ support small C0 mode and antiquant mode
11    if constexpr (enableSmallC0) {
12        DataCopyGM2L1ND2NZ<T, enableSmallC0>(dst, src, intriParams, enablePreReadIgnoreSync);
13        return;
14    }
15    const uint8_t cacheMode = ExtractCacheMode(src);
16    if (dstHWPos == Hardware::L1) {
17        // gm -> l1
18        DataCopyGM2L1ND2NZImpl((__cbuf__ PrimType*)dst.GetPhysAddr(), (__gm__ PrimType*)src.GetPhysAddr(),
19                                intriParams, cacheMode, enablePreReadIgnoreSync);
20    } else if (dstHWPos == Hardware::UB) {
21        DataCopyGM2UBND2NZImpl((__ubuf__ PrimType*)dst.GetPhysAddr(), (__gm__ PrimType*)src.GetPhysAddr(),
22                                intriParams, cacheMode);
23    } else {
24        ASCENDC_CHECK_TPOSITION(false, "dst", "A1 / B1 / VECIN",
25                                "DataCopy from GlobalTensor to LocalTensor with Nd2NzParams",
26                                ConstDefiner::Instance().logicNameMap.at(static_cast<uint8_t>(dst.GetPosition())));
27    }
28 }
```

去除数组成员，改用独立标量变量

```
// 反面：结构体内含动态索引数组
event_t eventIdV2Mte2[2] = {EVENT_ID0, EVENT_ID1};
SetFlag<HardEvent::V_MTE2> (
    eventIdV2Mte2[x1ScalePingPongID_]);

// 正面：拆为独立标量
uint16_t x1ScalePingPongID_ = 0;
SetFlag<HardEvent::V_MTE2> (x1ScalePingPongID_);
```

原理：动态下标 → 编译器假设最坏情况 → 对整个结构体放弃常量传播

现象：TPosition 是编译期常量，运行时却仍要动态判断 → 常量传播被破坏
25 组用例效果（节选 5 组代表）

#	优化前 SCALAR	优化后 SCALAR	SCALAR 降幅	TOTAL 提升
4	16,711,983	8,666,010	92.85%	8.23%
11	10,464,468	6,032,413	73.47%	7.40%
17	14,673,669	7,875,640	86.32%	6.21%
21	25,882,265	13,069,434	98.04%	25.69%
24	9,478,127	5,011,354	89.13%	11.55%

25 组平均

-50.15%

Scalar 平均降低

+10.37%

端到端平均提升

ScalarBound 完全消除 · 优化集中在「让编译器看懂常量」



① 减少内存访问

减少 Load/Store 指令数，降低寄存器 Spill

- 局部变量替代成员变量
- 静态创建 LocalTensor
- 消除多级指针

② 消除编译器优化障碍

帮助编译器完成常量传播 / 常量折叠

- 去除结构体中的动态索引数组
- 用值类型替代指针类型

③ 改善数据缓存

提高数据访问的空间 / 时间局部性

- 变量定义贴近使用
- 避免大结构体
- 减少数据拷贝

④ 改善指令缓存

消除 I-Cache miss，保证 Scalar 指令流连续

- icache 预取
- 代码精简
- 循环拆分减少代码体量

优先级建议：① 减少内存访问 → ② 消除编译器障碍 → ③ 数据缓存 → ④ 指令缓存

共同目标：帮助编译器更好地进行寄存器分配和常量传播 → 减少 Load/Store → 降低 ScalarBound 风险

7.1

结构体内慎用数组

动态下标访问数组会让别名分析失败，整个结构体的常量传播被放弃；拆为独立标量成员。

7.2

循环主尾块分离

把 Prologue / Hot Loop / Epilogue 分开，Hot Loop 保持零分支，避免热路径中分支预测失败被放大。

7.3

显式编写循环代码

用显式多层 for 替代隐式状态机：for 回跳走专用硬件不进分支预测，且编译器更容易做循环优化。

7.4

尽量使用局部变量

局部变量寄存器分配容易、别名分析友好、跨函数调用可保留在寄存器；成员变量必须经 this 访问、易被假设修改。

7.5

变量定义贴近使用位置

缩短 Live Range，减少寄存器占用时间，降低被 Spill 到栈的概率。

7.6

避免多级指针解引用

每多一级 = 一次 Load + 一条依赖链 + 一次潜在 D-Cache miss；用值类型聚合结构体替代。

7.7

避免使用超大结构体

单个结构体控制在 64B 内（最大 128B）；超大结构体打乱缓存、必栈分配，指针传递还引入别名问题。

一句话总结

写 Scalar 友好的代码 =
把更多决定权交给编译器
(更短的 Live Range、更少的指针、
更纯的结构体、更显式的循环)

目录

CONTENTS

Part 1 RmsNormQuant 性能深度优化

Part 2 Scalar 算子性能优化

Part 3 MXFP8/MXFP4 混合精度矩阵乘优化

MXFP8/MXFP4

混合精度矩阵乘优化实践

如何把 FP4 权重接入 FP8 矩阵乘 | Vector + Cube 混合路径 | 位操作优化

快 38.2%

M=1 算子耗时 17.4 us vs 28.2 us(MXFP8)

NRMSE 介于 MXFP8 与 MXFP4 之间

收益区间 $M \leq 128$

为什么需要 MXFP8/MXFP4

1

动机 · 权重搬运瓶颈

你做小 M 推理是不是也碰到过：Cube 核闲着，profiling 耗时全在 MTE2？卡你的不是算力，是权重搬运——这就是 MXFP8/MXFP4 想解决的东西。

MXFP8

FP8 / FP8

纯 Cube 核实现
权重 FP8 搬运
误差最小

MXFP8/MXFP4

FP8 / FP4

Vector+Cube混合核实现
权重 FP4 搬运 + Vector 预处理
折中路径

MXFP4

FP4 / FP4

纯 Cube 核实现
权重 FP4 搬运
性能最佳

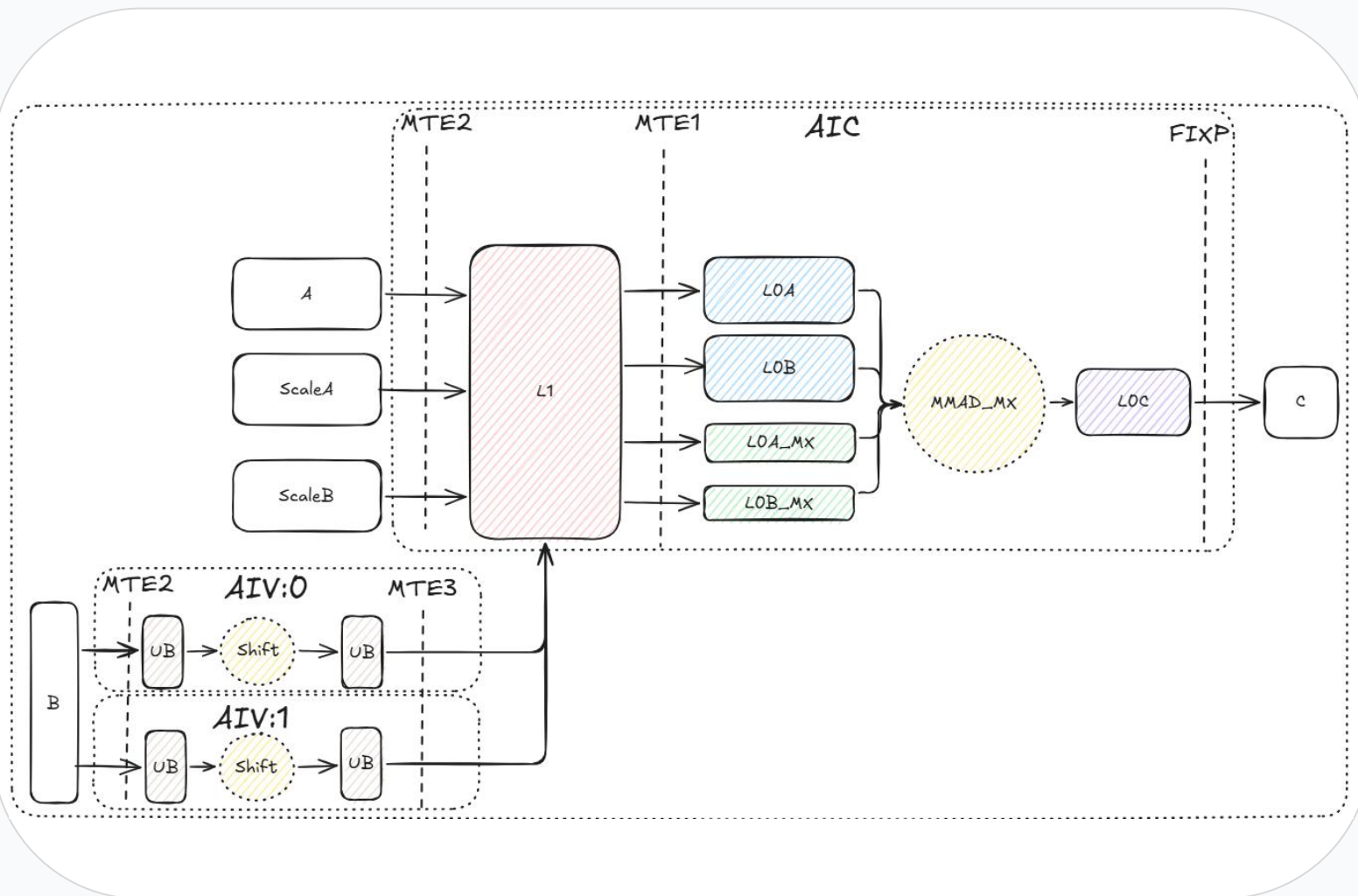
- 权重搬运量相比 MXFP8 减半。
- 净收益还要减去：Vector 核预处理、UB/L1 搬运、混合核同步调度开销。
- 三种量化模式需要的 scale dtype和shape一样 (float8_e8m0, groupsize=32) ；差异主要在权重的dtype和shape。

MXFP8/MXFP4混合核数据流

2

混合核数据流

$A / scaleA / scaleB$ 跟 MXFP8 对齐; 权重使用位操作进行预处理。



B路径的不同是造成性能差异的主要因素

MXFP8

B(GM) $\rightarrow L1 \rightarrow L0B \rightarrow L0C$

MXFP8/MXFP4

B(GM) $\rightarrow UB \rightarrow L1 \rightarrow L0B \rightarrow L0C$

有没有性能收益取决于:

1. FP4 节省的搬运是否覆盖 Vector 核预处理
2. 同步调度开销
3. 混合核开销

核心优化：多级cast vs 位操作

3

多级cast → 位操作 • ~85% VF↓

FP4 e2m1fn 转 FP8 e4m3fn/64, 本质是把 4bit 字段搬到 8bit 对应位置

✗ Cast 路线 • ~16条 / 完整处理

```
for subChunk ∈ {high, low}:    ← x2
  for k, inner:
    LoadAlign<UNPACK4_B8>      # 加载
    Cast FP4 → BF16
    MaskInterleave              # 重排
    Interleave                  # 重排
    Cast BF16 → FP32    x2      # 2 lanes
    Cast FP32 → FP8      x2      # 2 lanes
    DeInterleave                # 重排
    StoreAlign<PACK_B16>        # 存储
```

✓ 位操作路线 • 7 条 / 完整处理

```
for k, inner:
  LoadAlign<US_B8>              # 加载时上采样
  ShiftRight → wShr0             # 高4bit右移
  ShiftLeft  → wShl              # 低4bit左移
  ShiftRight → wShr1             # 同理处理低4bit
  Select     → wSel              # 拼接
  And        → wAnd              # 无关位置置0
  StoreAlign<NORM_B8>           # 存储
```

1 数据形态

FP4→BF16→FP32→FP8 3 跳精度转换

全程 int8 类型 · 无 dtype 转换

2 资源占用

6 个寄存器

5 个寄存器

实测验证：使用位操作后 VF 不再是瓶颈

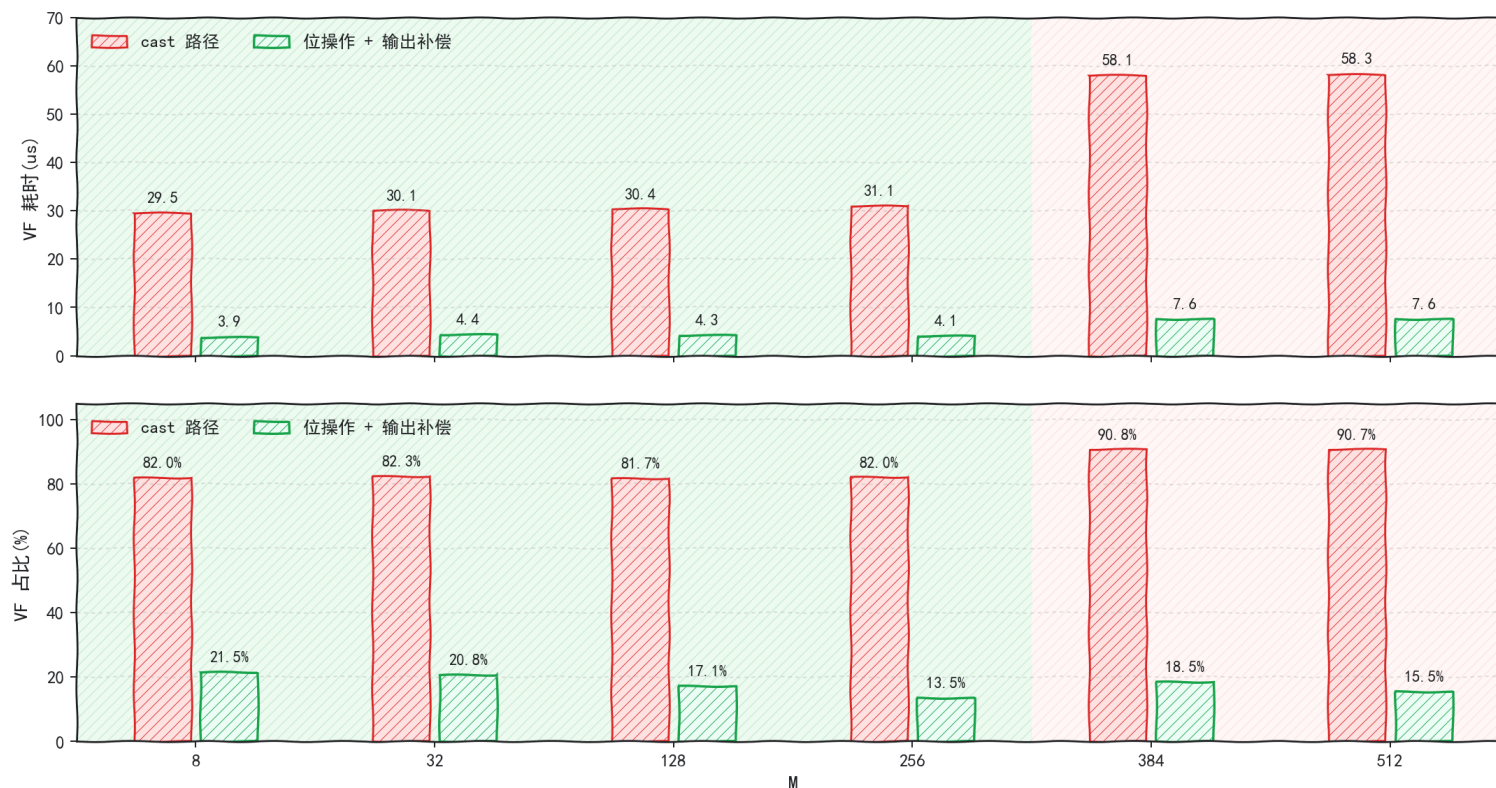
3

VF 29.5 $\rightarrow$ 3.9 μs $\cdot$ $\sim 7.5\times$

多级 cast 替换为位操作预处理后的收益

cast 改位操作：VF 耗时与 VF 占比对照

N=8192, K=4096; 上：VF 耗时(us); 下：VF 占 kernel 总耗时百分比



VF 耗时降到 $\sim 4 \mu\text{s}$, 不再是算子的瓶颈; 瓶颈成为 MTE2 和 MMAD。

性能收益：哪里有收益？哪里没收益？

4

性能区间 · $M \leq 128$ 收益

$M \leq 128$ 收益区间 · $M=256$ 拐点 · $M \geq 384$ 边界区

$M=1/2/4/8/16$

- 性能同 MXFP4 接近，比 MXFP8 快35%+
- 替换MXFP4无性能劣化，精度提升
- 替换MXFP8性能提升明显，精度劣化

$M=32/64/128/256$

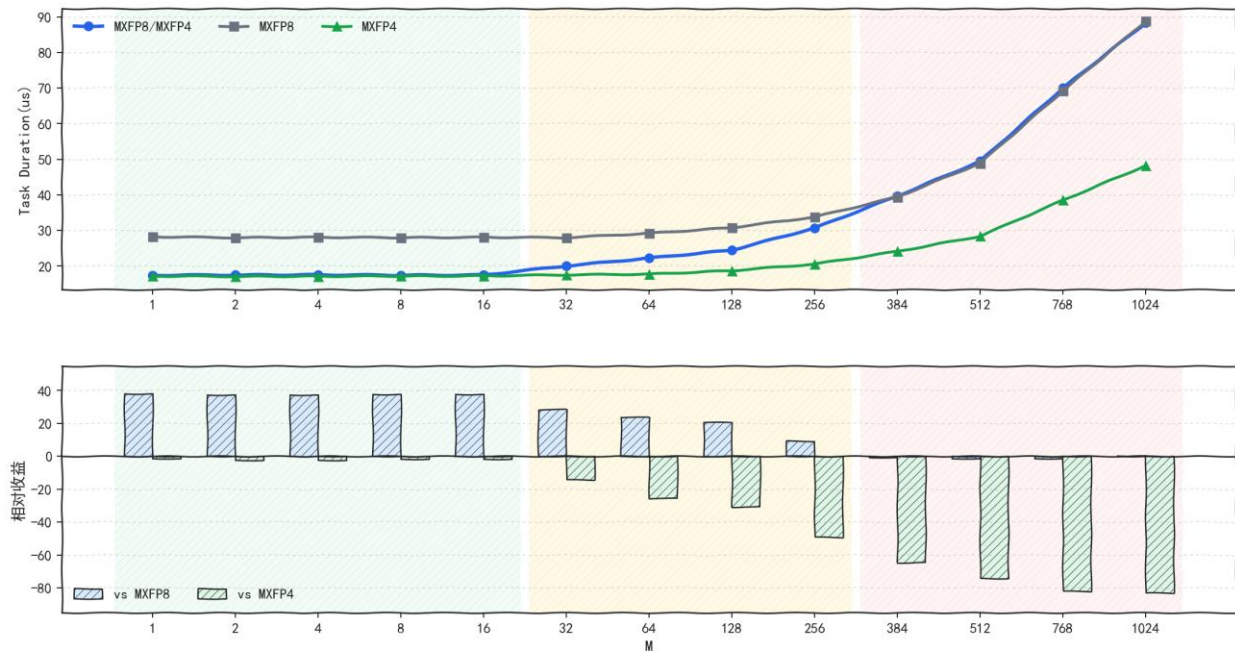
- 性能劣于MXFP4，但仍然比MXFP8快
- 替换MXFP4性能劣化，精度提升
- 替换MXFP8性能提升，精度劣化

$M \geq 384$

- 性能明显劣于MXFP4，同MXFP8近似
- 替换MXFP4性能劣化，精度提升
- 替换MXFP8性能不变，device内存占用少

K=4096, N=8192; kernel time 中位数 (us)

上: kernel 耗时; 下: MXFP8/MXFP4 相对基线收益



性能数据来自当前样例环境和固定 K/N; 不同芯片、CANN 版本、cache 状态和网络层需实测确认。NZ 指当前样例中的 NZ layout 基线。

数值误差：处在 MXFP8 和 MXFP4 之间

5

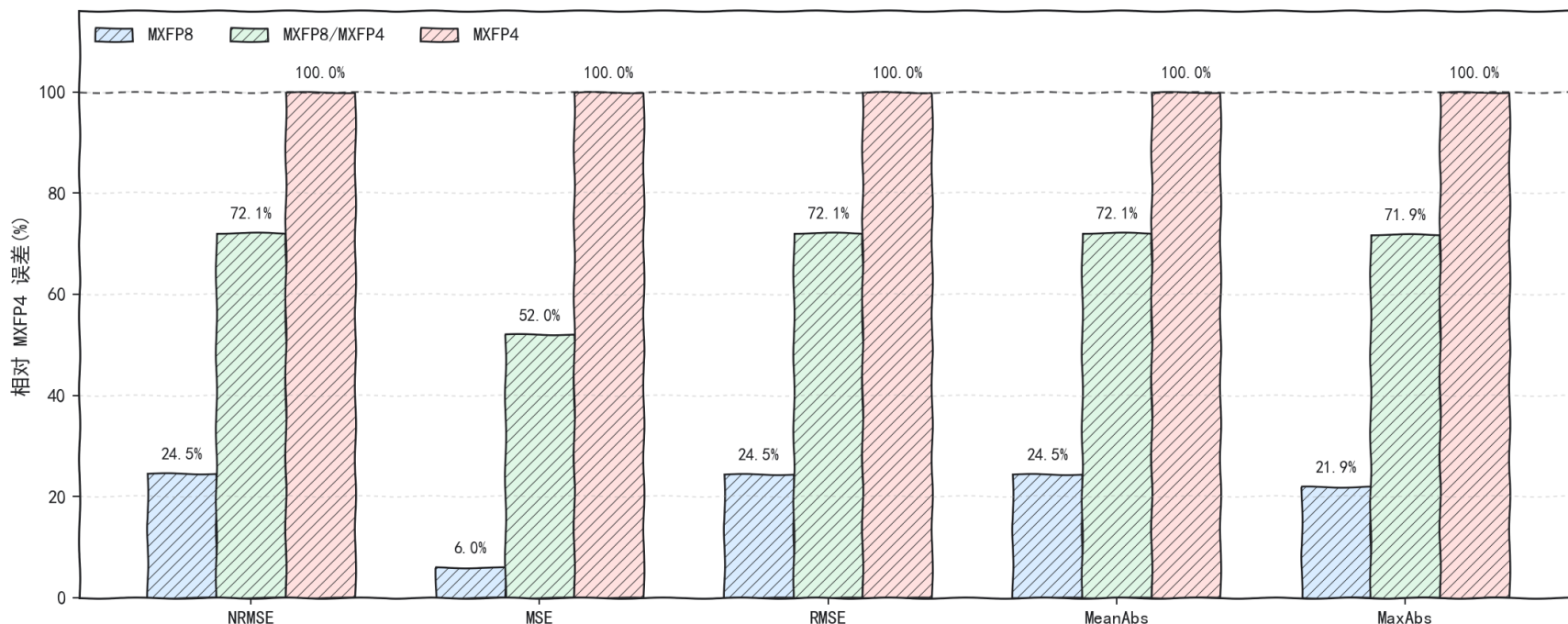
精度位置 • NRMSE 0.114

Golden = FP32 source matmul

为什么不直接选 MXFP4? 性能更强, 但 NRMSE 0.158——MXFP8/MXFP4 0.114 比 MXFP4 低 ~28%。

随机 FP32 输入下的 NPU 数值误差对比

Golden 为 FP32 source matmul; 以 MXFP4=100% 归一化, 数值越低越好。



验证方式:

1. 统一输入源
2. 做3种格式的量化
3. 采用多种误差指标比较, 数值越小, 误差越小

NRMSE (归一化均方根误差)
测试不同 shape 下的误差
MSE (均方误差) 对大误差更敏感
RMSE (均方根误差) 单位跟输出值一致
MeanAbs (平均绝对误差) 反映整体平均偏差
MaxAbs (最大绝对误差) 反应单点最大误差

Appendix: 位操作详细解释

数值公式:

MXFP4_E2M1(S:1,E:2,M:1)

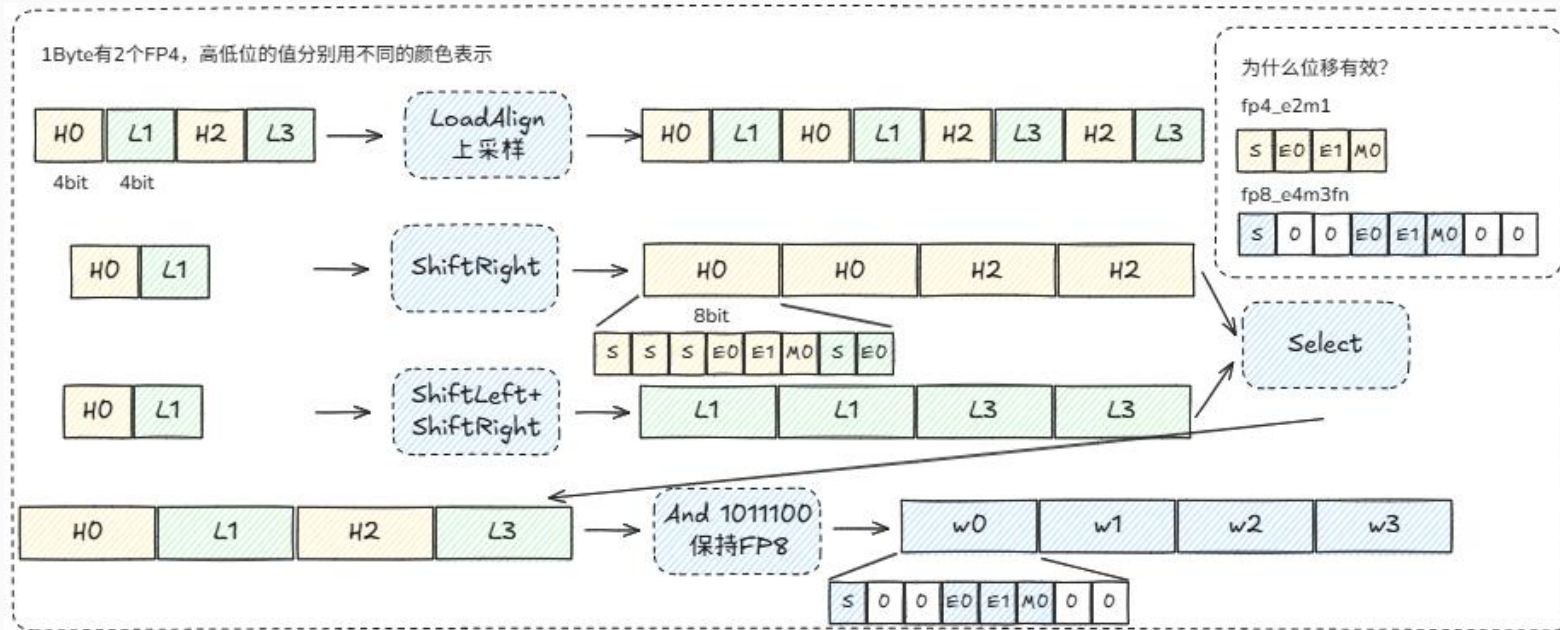
当指数位 $E \neq 0$: $v = (-1)^S \times 2^{E-1} \times (1.M)_2$

当指数位 $E = 0$: $v = (-1)^S \times 2^0 \times (0.M)_2$

MXFP8_E4M3FN(S:1,E:4,M:3)

当指数位 $E \neq 0$: $v = (-1)^S \times 2^{E-7} \times (1.M)_2$

当指数位 $E = 0$: $v = (-1)^S \times 2^{-6} \times (0.M)_2$



```
AscendC::Reg::LoadAlign<uint8_t, DIST_US_B8>(wLoad,  
weight4BitPhyAddr, aregWeightB8In);  
AscendC::Reg::ShiftRight(wShr0, wLoad, wShrReg, preg);  
AscendC::Reg::ShiftLeft(wShl, wLoad, wShlReg, preg);  
AscendC::Reg::ShiftRight(wShr1, wShl, wShrReg, preg);  
AscendC::Reg::Select(wSel, wShr1, wShr0, pregVsel);  
AscendC::Reg::And(wAnd, wSel, wAndReg, preg);
```

shift_w4_to_w8.h核心片段

(Samples/2_Performance/matmul_story/matmul_recipes/include/tile/shift_w4_to_w8.h)

正确性边界同 cast baseline; 其它 dtype / 特殊值 / 后处理顺序需单独验证。

Appendix: Bound 分型怎么落到字段

理论估算口径与 profiling 字段——对照

$$T_total_model = \max(T_MMAD, T_MTE2, T_VF, T_MTE1, T_MTE3, T_FIXPIPE)$$

当前 shape 主拐点在 MTE2 / MMAD (见下页对照) ; 其余 4 项备查。

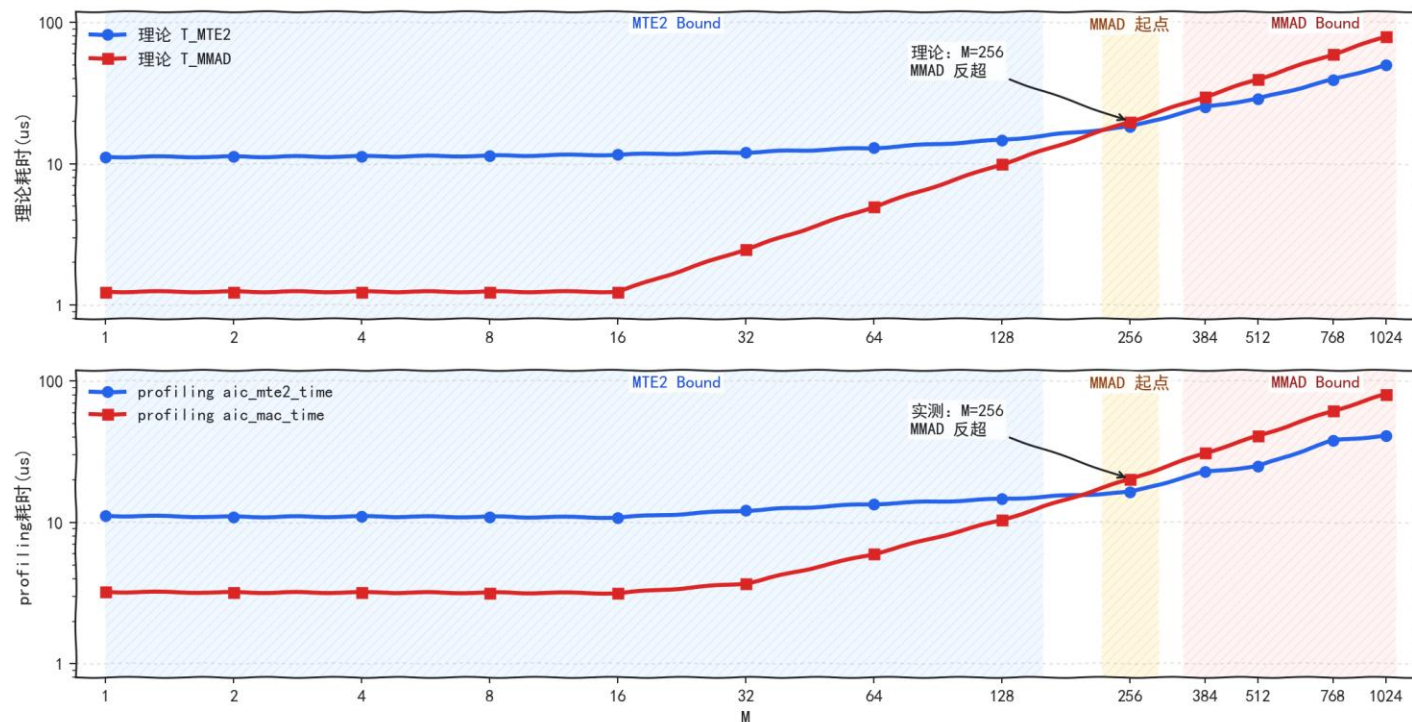
理论项	估算口径	profiling 对照
T_MMAD	$\text{align_up}(m,16) \cdot \text{align_up}(n,16) \cdot \text{align_up}(k,32) / (16 * 32 * 16 * \text{核数} * \text{频率})$	aic_mac_time
T_MTE2	$[\text{tileN} \cdot (m \cdot k + m \cdot k / 32) + \text{tileM} \cdot (n \cdot k \cdot 0.5 + n \cdot k / 32)] / \text{BandWidth_MTE2}$	aic_mte2_time / aiv_mte2_time
T_VF	Vector 核预处理工作量 / 有效吞吐	aiv_vec_time
T_MTE1	$\text{baseK} \cdot (1 + 1/32) \cdot (\text{baseM} + \text{baseN}) / \text{BandWidth_MTE1}$	aic_mte1_time / aic_mte1_ratio
T_MTE3	$k * n / \text{BandWidth_MTE3}$	aiv_mte3_time
T_FIXPIPE	$m * n * \text{sizeof}(\text{bfloat16}) / \text{BandWidth_FIXPIPE}$	aic_fixpipe_time

Appendix: 理论 Bound 与 profiling Bound 对应关系

同一组 M 点上，理论曲线和实测曲线在关键拐点一致

理论 Bound 与 profiling Bound 对照

上: 理论 T_{MTE2}/T_{MMAD} ; 下: profiling $aic_mte2_time/aic_mac_time$



$M \leq 128$ 理论 $T_{MTE2} > T_{MMAD}$

实测 $aic_mte2_time > aic_mac_time$

$M = 256$ 理论 T_{MMAD} 反超 T_{MTE2}

实测 aic_mac_time 反超 aic_mte2_time

$M \geq 384$ 理论 $T_{MMAD} > T_{MTE2}$

实测 $aic_mac_time > aic_mte2_time$

决定当前拐点的是 $MTE2 / MMAD$; 其余 4 项流水当前 shape 下未成为主瓶颈。

小结

路径

FP4 → Vector 核预处理 → Cube 核 MMAD_MX;
不是简单 dtype 替换

优化

位操作替代多级 cast 保住 权重搬运收益; 输出侧 ×64 补偿

性能边界

$M \leq 128$ 相比 MXFP8 NZ 有收益; $M=256$ 拐点;
 $M \geq 384$ 性能无收益

精度位置

全局 NRMSE 0.114; 低于 MXFP4 (0.158), 高于 MXFP8 (0.039)

行动

打开cann-samples
替换 shape 跑 MXFP8 NZ / MXFP8/MXFP4 / MXFP4 NZ 三组对比
同时看 kernel time 与 NRMSE

Issue / PR • 分享 shape、瓶颈拐点、混合核调度经验

Thank you.

社区愿景：打造开放易用、技术领先的AI算力新生态

社区使命：使能开发者基于CANN社区自主研究创新，构筑根深叶茂、跨产业协同共享共赢的CANN生态

Vision: Building an Open, Easy-to-Use, and Technology-leading AI Computing Ecosystem

Mission: Enable developers to independently research and innovate based on the CANN community and build a win-win CANN ecosystem with deep roots and cross-industry collaboration and sharing.



上CANN社区获取干货



关注CANN公众号获取资讯